

BENNETT UNIVERSITY

School of Computer Science Engineering and Technology

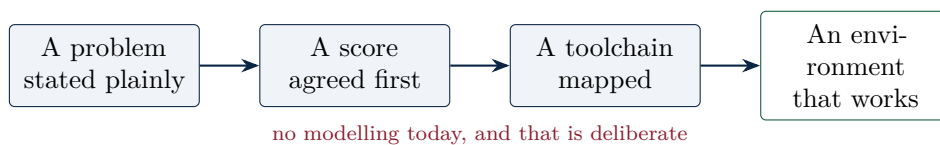
B.Tech Computer Science and Engineering, Semester V

SCSE3040 • Machine Learning Operations

Lecture 02

Meet the Project

One model, all semester: a problem, a score,
a toolchain and a place to type



Recap of Lecture 01 in three points. A model in a notebook is not a product. The lifecycle is a loop, and monitoring feeds back into data. MLOps borrows DevOps habits, then adds data and models.

We agreed on a destination last time. We never said what we are actually building. Today we do.

Contents

How this handout maps to the approved session plan	4
I The Promise, Not the Parcel	5
1 What the dabbawala actually sells	5
2 The project, stated plainly	5
3 The data we will use	6
II Agree the Score Before You Build	8
4 Mean absolute error, defined properly	8
5 The alternatives, and what each one punishes	8
5.1 A worked example you can do on paper	8
5.2 So which do we choose, and why	9
6 The number to beat: baselines before models	9
7 One number is never enough: slices	10
III Eight Tools, One Job Each	11
8 The map	11
9 The arc we will walk	12
10 Common errors in reading the map	12
IV Where Your Code Lives and Runs	13
11 Two places you can run code	13
12 Why a virtual environment	13
13 Where your code will live	13
14 Set this up before Lecture 03	14
15 What our project gains today	14

V Consolidation	15
16 Assignment questions, as released	15
17 Check your understanding	15
18 Vocabulary introduced today	16
VI The Week Ahead	17
19 Seven days, about seven hours	17
VII Resource Library	19
20 For this lecture specifically	19
21 Carried forward from Lecture 01	19
22 Toolchain manuals, for when each session arrives	20

List of Figures

1	The prediction is not the product. The promise is the product, and the prediction is how the promise gets made.	5
2	The problem framing card. Answer these five before writing a line of modelling code, and keep the answers in the repository where a colleague can read them.	6
3	Identical MAE, very different behaviour. MAE adds the errors; RMSE squares them first, so the single large error dominates. Choosing between them is a statement about which failure you can live with, not a statistical technicality.	9
4	The baseline ladder. A score with nothing to compare it against is not evidence, and the baseline is what turns MAE from a number into a judgement.	9
5	The semester arc, with the versioning layer restored underneath. The first four boxes make the model callable; the second four make it something a company could operate.	12
6	Two places, one project. The ephemeral runtime is the whole point of the distinction: anything that must survive being closed does not belong only in Colab.	13
7	The single most common cause of it worked yesterday. The environment is not an optional tidiness measure; it is the first of the three reproducibility habits from Practical 01.	13
8	A minimal but standard layout. The separation of notebooks from source is not cosmetic: exploration and shipped code have different lifetimes, and Lecture 07 is about moving one into the other.	14
9	Where we stand at the end of Lecture 02. Nothing has been trained yet, and that is the correct order of work.	14

How this handout maps to the approved session plan

The released deck for this lecture consolidates material that the session plan distributes across several entries. That is a sensible teaching decision, because students cannot start Lab 1 without the project definition and the environment. The mapping below is the honest account.

Topic in this handout	Session plan location	Note
The running project methodology, notebook to production	Session 1	Introduced in Lecture 01, developed fully here because Lab 1 depends on it
Selecting and justifying the right metric for delivery time prediction	Session 5, the named activity	Brought forward, so that the score is agreed before the first model rather than after
The tool map, one job per tool	Sessions 1 and 2, principles	The map only; each tool gets its own session later
Reproducible environments, Colab and local	Lab 1, sessions 3 and 4	Lecture support for the laboratory that follows
Version control philosophy, what belongs in Git	Session 9	Preview only. The full treatment is Session 9, and data versioning is Session 10

One scope correction to state in class

The arc in the released deck names Kubernetes as one of the eight tools. In the approved plan Kubernetes is *conceptual only*, theory without laboratory work, and Docker Compose is the tool this course actually uses for multi service orchestration. Say this once, at the tool map, so nobody prepares for a laboratory that does not exist.

The taught hour, minute by minute

0 to 6	Recap of Lecture 01 in three points. Then the dabbawala, Section 1
6 to 16	Sections 2 and 3, the project stated plainly and the data dictionary
16 to 30	Sections 4 to 6, agreeing the score. Do the six row worked example live on the board
30 to 38	Section 7, baselines, and Section 8, slices. This is where the deck's assignment Q2 comes from
38 to 48	Sections 9 and 10, the tool map and the arc, with the Kubernetes correction
48 to 56	Sections 11 to 13, Colab against laptop, and the virtual environment
56 to 60	Section 14, the setup commands, and what to have working before Lecture 03

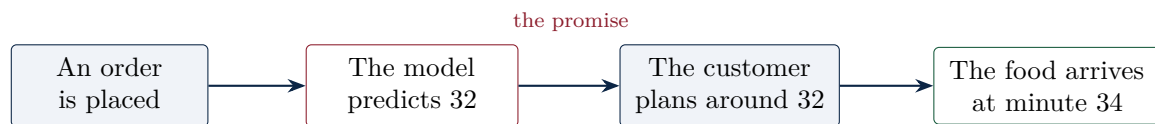
PART I

The Promise, Not the Parcel

1 What the dabbawala actually sells

A Mumbai dabbawala carries your lunch box across the city. What you pay for is not the box being moved. You pay for the promise that it arrives by 12:45.

Food delivery applications sell exactly the same promise, and they state it as a number: arriving in 32 minutes. That number is a model's output. Get it wrong and customers leave, not because the food was late in an absolute sense, but because the promise was broken.



The engineering question is not whether 34 is close to 32. It is what the business loses each time the promise is broken, how often that happens, and who finds out first: the dashboard or the customer.

Figure 1: The prediction is not the product. The promise is the product, and the prediction is how the promise gets made.

2 The project, stated plainly

The delivery time predictor

Given one food order, predict how many **minutes** until it reaches the customer.

Who uses it The application, at the moment of ordering

How often Every single order, thousands a day

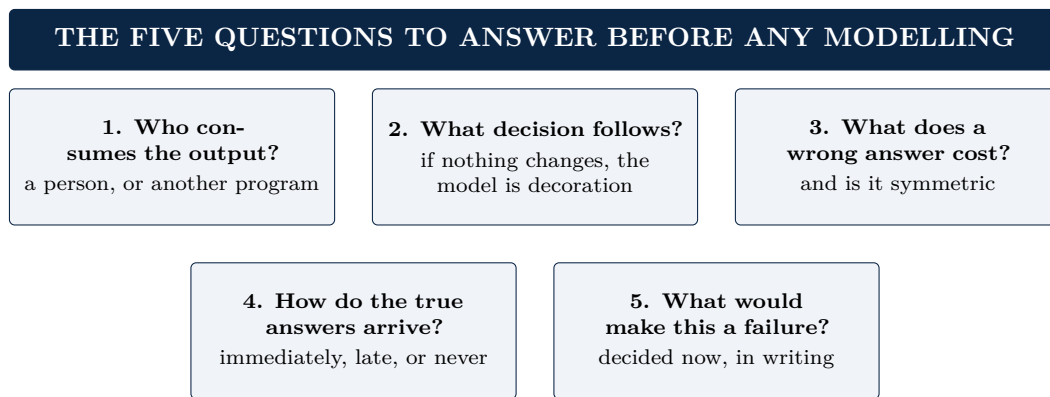
How fast An answer in well under a second

Output type A single number, in minutes

Consequence The number is shown to a customer and becomes a commitment

We build this one project, and only this one, for the whole semester.

Notice how much of that specification has nothing to do with machine learning. Latency, frequency, consumer and consequence are engineering facts, and they constrain every later choice: which model families are viable, whether a batch job is acceptable, how much a wrong answer costs, and what has to be monitored.



Question 4 is the one students skip, and it decides the entire monitoring design later in the course. For delivery times the answer is comfortable: the truth arrives about half an hour later. For fraud, or for medical prognosis, it may take months or never arrive at all.

Figure 2: The problem framing card. Answer these five before writing a line of modelling code, and keep the answers in the repository where a colleague can read them.

3 The data we will use

Column	Units	Meaning	What could go wrong with it
distance_km	km	How far the restaurant is from the customer	Straight line against road distance; a silent change in how it is computed shifts every prediction
prep_time_min	minutes	How long the kitchen takes	Often estimated rather than measured, so it may itself be a model output. That is a correction cascade waiting to happen
traffic_level	1, 2, 3	1 light, 2 medium, 3 heavy	An ordered category encoded as a number. A new level 4 appearing later breaks assumptions silently
rain	0 or 1	0 dry, 1 raining	Binary today. If the source later reports millimetres, the column type changes underneath you
delivery_min	minutes	What we predict	The label. Recorded when the rider marks delivered, which is a human action and therefore noisy

Four inputs, one output, deliberately small. The hard part of this course is never the model, and a small dataset keeps every laboratory fast on an 8 GB machine.

A habit worth starting today

That fourth column of the table is called a *data dictionary with failure notes*, and almost no student project has one. Writing it takes twenty minutes and it is the single cheapest defence against the data failures that ended three of the four cases in Lecture 01.

PART II

Agree the Score Before You Build

4 Mean absolute error, defined properly

For n orders, with true delivery times y_i and predictions $\hat{y}_i$:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Read it in the units of the problem. An MAE of 3 means: typically wrong by about three minutes. Lower is better. A customer forgives three minutes. A customer does not forgive twenty.

The habit, not the formula

Agreeing the score *before* you build is an MLOps habit rather than a formality. Two consequences follow immediately. Your teammates can no longer argue about whether a change was an improvement, because there is one number. And you can automate the decision later: a continuous integration pipeline can only block a bad model if somebody has already written down what bad means.

5 The alternatives, and what each one punishes

Metric	What it computes	What it punishes	When to prefer it
MAE	Mean of absolute errors	All errors equally, minute for minute	You want a number a customer would recognise
RMSE	Square root of the mean of squared errors	Large errors much more than small ones	One catastrophic error matters more than many small ones
MAPE	Mean of absolute percentage errors	Errors on small true values enormously	Comparing across items of very different scale, and never when the truth can be near zero
R^2	Proportion of variance explained	Nothing, in the units of the problem	Explaining fit to a statistician, not promising a time to a customer

5.1 A worked example you can do on paper

Six orders. The same six errors under two metrics.

Order	1	2	3	4	5	6	MAE	RMSE
True y	30	25	40	35	28	45		
Model A predicts	32	27	38	33	30	43	2.00	2.00
Model B predicts	30	25	40	35	28	33	2.00	4.90

Model A is wrong by two minutes on every order. Model B is perfect on five orders and twelve minutes wrong on the sixth. **Their MAE is identical. Their RMSE is not.**

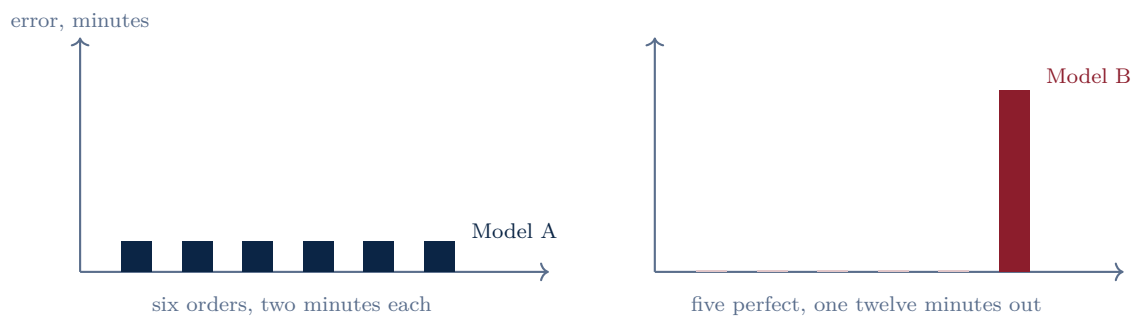


Figure 3: Identical MAE, very different behaviour. MAE adds the errors; RMSE squares them first, so the single large error dominates. Choosing between them is a statement about which failure you can live with, not a statistical technicality.

5.2 So which do we choose, and why

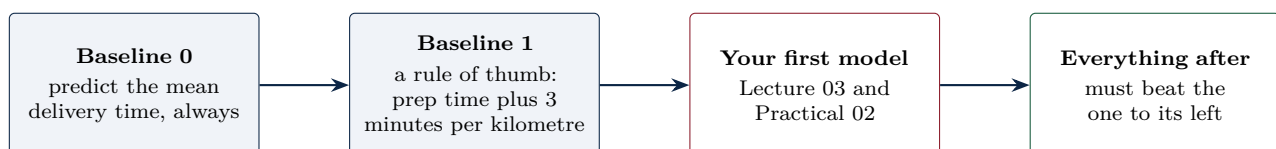
For this project we use **MAE, in minutes**, for three reasons. It is in the units of the promise, so a non technical stakeholder can read it. Delivery times contain genuine outliers, a stuck rider, a closed road, and we do not want the score dominated by events the model could never have predicted. And it is stable enough to compare across the whole semester, which matters because one number will follow this project through thirteen practicals.

The subtlety worth raising with a strong class

MAE treats a five minute early prediction and a five minute late prediction as equally bad. For a delivery promise they are not equally bad: arriving early is a pleasant surprise, arriving late is a broken promise. A metric that punishes lateness more than earliness is called an asymmetric or quantile loss, and it is what a real delivery platform would use. We stay with MAE for teaching clarity, and you should be able to explain in a viva why a production team might not.

6 The number to beat: baselines before models

Before any model, compute what a trivial rule achieves. Two are enough.



If your model cannot beat the rule of thumb, the honest engineering answer is to ship the rule of thumb. Google's own guidance opens by advising exactly this: do not be afraid to launch a product without machine learning, and keep the first model simple while getting the infrastructure right.

Figure 4: The baseline ladder. A score with nothing to compare it against is not evidence, and the baseline is what turns MAE from a number into a judgement.

7 One number is never enough: slices

Report the MAE overall, and then report it again on the parts of the data you care about.

Slice	MAE	What it tells you
All orders	3.0	The headline number, and the one that hides things
Dry weather	2.4	The common case, where most of the data lives
Raining	9.1	The case that generates complaints, invisible in the headline
Distance over 8 km	6.7	A systematic weakness worth a feature, not an apology

The question from the assignment, and how to think about it

Two models are tested on the same orders. Model A has MAE 4 minutes. Model B has MAE 3 minutes but is wrong by 40 minutes on rainy evenings. Which would you ship?

There is no single right answer, which is the point. A defensible answer names the slice, quantifies how often it occurs, and states the business consequence: if rainy evenings are two percent of orders but thirty percent of complaints, model B's better average is bought with the customers you can least afford to lose. A weak answer compares only the two headline numbers.

PART III

Eight Tools, One Job Each

8 The map

You will meet many names this semester. They are not competing. Each does one job in the loop from Lecture 01, each gets its own session later, and today is only the map.

A kitchen has a knife, a pan and a fridge. Nobody asks which is best.

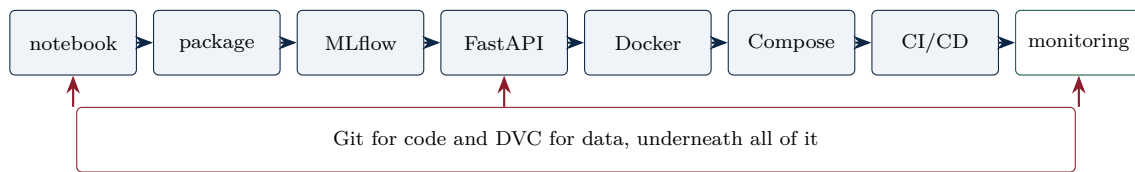
Tool	The one job it does	What it holds for our project	Official manual
Notebook	Where you explore and first get a model	The first delivery time model and its MAE	jupyter.org
Package	The same code, tidied into files others can run and test	<code>train.py</code> , <code>config.yaml</code> , tests	docs.python.org
MLflow	Remembers every training attempt and its score	Every run, its parameters and its MAE	mlflow.org
FastAPI	Wraps the model so other software can call it	A <code>/predict</code> endpoint returning minutes	fastapi.tiangolo.com
Docker	Packs the service so it runs the same anywhere	An image containing the model and the API	docs.docker.com
Docker Compose	Runs several services together on one network	The tracking server and the API side by side	docs.docker.com
GitHub Actions	Tests and ships changes automatically	A pipeline that runs the tests on every push	docs.github.com
Evidently	Tells you when the answers get worse	A drift report over recent orders	docs.evidentlyai.com

Two corrections to the deck's list, per the approved plan

Kubernetes appears on the released arc. In this course it is conceptual only, for industry literacy, and **Docker Compose** is the tool you will actually use for running several services together.

Git and **DVC** do not appear on the arc at all, yet they are Module I of the syllabus and Practicals 01 and 03. Code versioning and data versioning are the ninth and tenth jobs, and they sit underneath everything else on the map.

9 The arc we will walk



Same project, moving right. Every session pushes it one box along. After FastAPI the model is callable; after monitoring it is a product.

Figure 5: The semester arc, with the versioning layer restored underneath. The first four boxes make the model callable; the second four make it something a company could operate.

10 Common errors in reading the map

The error	The correction
Treating tools as rivals	Docker does not replace FastAPI. They sit next to each other: one wraps the model in an interface, the other packs the result so it runs anywhere
Learning the commands, not the job	You will forget the flags within a month. Remember which problem each tool solves, and the manual will give you the flags back in thirty seconds
Skipping ahead to the impressive names	Without the first four boxes there is nothing to orchestrate. An empty cluster is not an achievement
Assuming one tool covers two jobs	A teammate who says we use Docker so we do not need MLflow has confused packaging with remembering. Neither does the other's job

PART IV

Where Your Code Lives and Runs

11 Two places you can run code

Nothing to install, runs in a browser
 Opens a notebook in seconds
 Free, and enough for early modelling
But the machine is thrown away
 when you close the session
Correct for exploring and first models, Lectures 03 to 06

YOUR OWN LAPTOP

Python installed locally
 A virtual environment per project
 Files persist, and so does your work
 Needed the moment Docker,
 services and pipelines appear

Correct from Lecture 07 onward, and for every laboratory

You will use both, for different halves of the course

Figure 6: Two places, one project. The ephemeral runtime is the whole point of the distinction: anything that must survive being closed does not belong only in Colab.

12 Why a virtual environment

Definition

A **virtual environment** is a private box of libraries for one project, so that two projects on the same laptop cannot break each other.

Like separate dabbas in one tiffin carrier. Same carrier, nothing mixes.

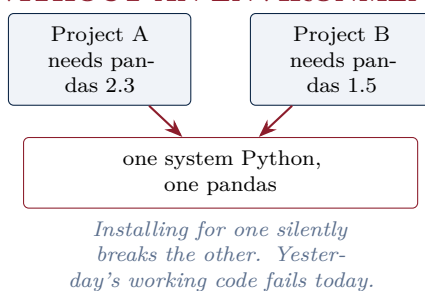
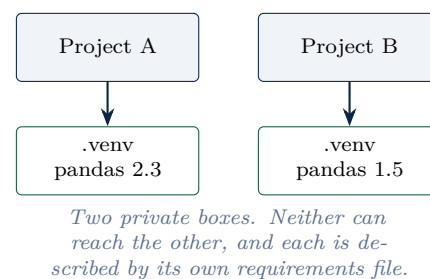
WITHOUT AN ENVIRONMENT**WITH ENVIRONMENTS**

Figure 7: The single most common cause of it worked yesterday. The environment is not an optional tidiness measure; it is the first of the three reproducibility habits from Practical 01.

13 Where your code will live

The starter kit lives in a Git repository. You clone it once, then commit your work as you go. Cloning copies the whole history, not just today's files, which is why a repository is a record rather than a folder.

```
mlops-starter-kit/
  README.md
  requirements.txt
  .gitignore
  data/
  src/
  notebooks/
  tests/
  .venv/
```

what the project is, and how to run it
the exact libraries, pinned
what Git must ignore
the delivery dataset, tracked by DVC later
the code that becomes a package in Lecture 07
exploration, which is not the same as the package
the tests that make a green pipeline possible
your private libraries, and never committed

data/ and .venv/
 never enter the repository

Figure 8: A minimal but standard layout. The separation of notebooks from source is not cosmetic: exploration and shipped code have different lifetimes, and Lecture 07 is about moving one into the other.

The global standard worth reading this week

Cookiecutter Data Science by DrivenData is the most widely adopted opinionated layout for data projects, and its *opinions* page explains *why* each folder exists rather than just listing them. <https://cookiecutter-data-science.drivendata.org/>

14 Set this up before Lecture 03

```
git clone <starter-kit-url>
cd mlops-starter-kit
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -c "import pandas, sklearn; print('ready')"
```

On macOS or Linux, use python3 and activate with `source .venv/bin/activate`. The repository link is on the course LMS. If the last line prints `ready`, you are done.

Common errors with environments

- **Colab here, laptop there, different versions.** Then code that ran yesterday fails today. Always install from `requirements.txt`, never by memory.
- **Committing .venv or the dataset to Git.** The repository becomes enormous and slow, and Git is the wrong tool for both. Use `.gitignore`, and wait for DVC in Practical 03.
- **Forgetting to activate.** You install into the system Python, then wonder why the import fails. If `pip install pandas` succeeds and `import pandas` fails, this is almost always the cause.

15 What our project gains today



The project now has a definition and a home: a stated problem, four known input columns, an agreed score in minutes, and a repository with a working environment to put code in. Next lecture we finally put a model inside it.

Figure 9: Where we stand at the end of Lecture 02. Nothing has been trained yet, and that is the correct order of work.

PART V

Consolidation

16 Assignment questions, as released

Set 1 • The running project • CO1

Q1 (Understand). State the delivery time problem in your own words: what goes in, what comes out, and who consumes the answer. Then explain what MAE measures and why lower is better.

Q2 (Apply). Two models are tested on the same orders. Model A has MAE 4 minutes; model B has MAE 3 minutes but is wrong by 40 minutes on rainy evenings. Say which you would ship and why. There is no single right answer, so defend yours in three sentences.

Q3 (Build). Write a one page project brief for the delivery time predictor. It must contain the problem in two sentences, the four input columns with the units of each, the score you will use, and one sentence on what would make you call the project a failure.

Set 2 • The tools map • CO1

Q1 (Understand). List the eight tools on the arc in order, and give the one job each does in a single sentence. Then name the two that would still be needed if the project had only one user.

Q2 (Apply). A teammate says we already use Docker, so we do not need MLflow. Explain in three or four sentences why this confuses two different jobs, and name what each tool would still not do for you.

Q3 (Build). Redraw the arc for the delivery time project on one page. Under each box, write one sentence on what that tool will specifically hold or do for this project. Mark the two boxes you understand least.

Set 3 • Your environment • CO2

Q1 (Understand). Explain what a virtual environment is and what problem it solves. Then give one situation where Colab is the better choice, and one where your own laptop is.

Q2 (Apply). A classmate runs `pip install pandas`, then opens Python and gets `ModuleNotFoundError: pandas`. Give the most likely cause, the exact commands you would ask them to run to confirm it, and the fix.

Q3 (Build). Set up your machine: clone the starter kit, create and activate a virtual environment, install from `requirements.txt`, and confirm the check line prints `ready`. Then add a `.gitignore` that excludes `.venv` and any `.csv` data, and commit it. Submit the output of `pip list` as proof.

17 Check your understanding

1. What is the dabbawala actually selling, and what is the equivalent in our project?
2. Which parts of the project specification have nothing to do with machine learning, and why do they still constrain the model?
3. Write the MAE formula from memory, then say what an MAE of 3 means to a customer.
4. Two models have the same MAE but very different RMSE. What does that tell you about their

error patterns?

5. Why is MAPE a poor choice when the true value can be near zero?
6. Give the two baselines this project uses, and say why a model that cannot beat the second should not ship.
7. Give one slice of our data where a good headline MAE could be hiding a serious problem.
8. Of the five framing questions, which one decides the monitoring design, and what is the answer for delivery times?
9. Name the one job done by each of MLflow, FastAPI, Docker and Evidently.
10. Why does this course use Docker Compose rather than Kubernetes for laboratory work?
11. What is thrown away when you close a Colab session, and what follows for where your project lives?
12. Give the three things that must never be committed to the course repository, and say why for each.

18 Vocabulary introduced today

Term	Meaning
MAE, mean absolute error	The average size of the error, in the units of the target. Lower is better
RMSE, root mean squared error	The square root of the average squared error. Punishes large errors much more than small ones
MAPE	Mean absolute percentage error. Unstable when true values approach zero
Baseline	A trivial rule whose score any model must beat before it is worth deploying
Slice	A named subgroup of the data on which the metric is reported separately
Data dictionary	A written description of every column: meaning, units, and how it can go wrong
Label	The true value the model is trying to predict, recorded after the fact
Virtual environment	A private set of libraries for one project
Requirements file	The pinned list of libraries that rebuilds that environment exactly
Repository	A folder plus its complete history, cloned rather than copied
Ephemeral runtime	A machine that is discarded when the session ends, such as a Colab instance

PART VI

The Week Ahead

19 Seven days, about seven hours

Each day names one resource, the time it takes, and one artefact you produce. Keep the outputs together; three of them become part of your project brief.

Day 1 • Metrics, properly • 60 minutes

Read. The metrics and scoring sections of the scikit-learn user guide. https://scikit-learn.org/stable/user_guide.html

Produce. Compute MAE and RMSE by hand on ten rows of the delivery data, then reproduce both with scikit-learn and confirm they match. If they do not, you have found a bug in your understanding, which is the cheapest kind to fix.

Day 2 • How forecasters think about accuracy • 60 minutes

Read. Hyndman and Athanasopoulos, *Forecasting: Principles and Practice*, third edition, the chapter on evaluating point forecast accuracy. Free and continuously updated. <https://otexts.com/fpp3/>

Why. Forecasting has thought harder about error measures than machine learning has. You will meet scale dependent errors, percentage errors and scaled errors, and you will understand why the choice of metric is a modelling decision.

Produce. One paragraph on why percentage errors fail for quantities that can be near zero.

Day 3 • Baselines and judgement • 45 minutes

Read. Google's *Rules of Machine Learning*, rules 1 to 4, again. <https://developers.google.com/machine-learning/guides/rules-of-ml>

Produce. Write the rule of thumb baseline for delivery time as three lines of Python, and record its MAE. That number is what your first model must beat in Lecture 03.

Day 4 • Your two workbenches • 60 minutes

Do. Open Colab and run one cell. <https://colab.research.google.com/> Then read the official venv documentation and build the local environment. <https://docs.python.org/3/library/venv.html>

Produce. The output of `pip list` from your activated environment, saved to a file. This is also assignment Set 3, Q3.

Day 5 • How professionals lay out a project • 45 minutes

Read. Cookiecutter Data Science, the structure and the opinions pages. <https://cookiecutter-data-science.drivendata.org/>

Produce. Map our starter kit onto their layout. Note two folders they have that we do not, and say what each would hold for the delivery project.

Day 6 • Git as a record, not a backup • 60 minutes

Read. Pro Git, chapter 2, Git basics. Free. <https://git-scm.com/book/en/v2>

Also. GitHub documentation on cloning a repository. <https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository>

Produce. A `.gitignore` for the project that excludes the environment and the data, committed with a message that explains why rather than what.

Day 7 • Write the brief • 60 minutes

Do. Assignment Set 1, Q3: the one page project brief. Use the five framing questions from Figure 2 as your headings, and include the data dictionary with failure notes from Section 3.

Why this matters beyond the marks. A project brief is the first artefact a real team produces, and the one that prevents an entire semester of arguing about what was being built.

If you only have one hour all week

Day 3, then Day 4. A baseline with a recorded number, and a working environment. Those two are what Lecture 03 assumes you arrive with.

PART VII

Resource Library

All links verified on 21 August 2026.

20 For this lecture specifically

Resource	Where, and why
Hyndman and Athanasopoulos, <i>Forecasting: Principles and Practice</i> , third edition	https://otexts.com/fpp3/ The best free treatment of error measures anywhere
Google, <i>Rules of Machine Learning</i>	https://developers.google.com/machine-learning/guides/rules-of-ml Rules 1 to 4 are about exactly today's material: heuristics, metrics and infrastructure first
Cookiecutter Data Science, DrivenData	https://cookiecutter-data-science.drivendata.org/ The reference project layout, with the reasoning
scikit-learn user guide	https://scikit-learn.org/stable/user_guide.html Metrics and scoring, and the model selection map
Google Colab	https://colab.research.google.com/ The zero setup notebook, with an ephemeral machine
Python venv documentation	https://docs.python.org/3/library/venv.html The official account of what an environment is
pip requirements file format	https://pip.pypa.io/en/stable/reference/requirements-file-format/ Why pinning with a double equals matters
Pro Git, free book	https://git-scm.com/book/en/v2 Chapter 2 is all you need this week
GitHub, cloning a repository	https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository

21 Carried forward from Lecture 01

Resource	Where
Sculley et al., <i>Hidden Technical Debt in Machine Learning Systems</i> , NeurIPS 2015	https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf

Resource	Where
Sato, Wider and Windheuser, <i>Continuous Delivery for Machine Learning</i>	https://martinfowler.com/articles/cd4ml.html
Amershi et al., <i>Software Engineering for Machine Learning</i> , ICSE 2019	https://www.microsoft.com/en-us/research/wp-content/uploads/2019/03/amershi-icse-2019_Software_Engineering_for_Machine_Learning.pdf
Paleyes, Urma and Lawrence, <i>Challenges in Deploying Machine Learning</i>	https://arxiv.org/abs/2011.09926
Google Cloud, MLOps maturity model	https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning
Machine Learning in Production, DeepLearning.AI, free to audit	https://www.coursera.org/learn/introduction-to-machine-learning-in-production
MLOps Zoomcamp, DataTalks.Club	https://github.com/DataTalksClub/mlops-zoomcamp
Made With ML	https://madewithml.com
Stanford CS 329S	https://stanford-cs329s.github.io

22 Toolchain manuals, for when each session arrives

Tool	Official manual
DVC	https://doc.dvc.org/start
pytest	https://docs.pytest.org/en/stable/getting-started.html
MLflow	https://mlflow.org/docs/latest/ml/getting-started/
FastAPI	https://fastapi.tiangolo.com
Docker	https://docs.docker.com/get-started/
GitHub Actions	https://docs.github.com/en/actions/get-started/quickstart
Apache Airflow	https://airflow.apache.org/docs/apache-airflow/stable/start.html
Evidently	https://docs.evidentlyai.com/quickstart_ml
Kubernetes, concepts only	https://kubernetes.io/docs/concepts/

Agree the promise before you build the model.

Next lecture. Lecture 03, machine learning refresher I: regression against classification and which one our problem is, the metrics MAE and RMSE compared on real predictions, and a first model on the delivery time data in scikit-learn. Come with your environment already working.